AI-BASED JOB RESUME SCREENING SYSTEM

Bala Sankar R (Department of CSE) 24619126@vistas.in
Arjun (Department of CSE) arjunnehru2006@gmail.com
Jotheshwar (Department of CSE) eshwareshwar82210@gmail.com
Department of Artificial Intelligence and Machine Learning
College Project Research Paper

Abstract— Hiring teams may receive a large number of resumes for one position. Manual first-level screening can be repetitive and can make consistent comparison difficult. This paper presents an AI-based job resume screening system that accepts a resume and a job description, extracts useful text, identifies relevant skills and qualifications, and produces an interpretable matching score. The proposed implementation uses conventional Natural Language Processing (NLP) and machine-learning methods rather than a large language model. Resume text is cleaned and converted into TF-IDF features, while a Support Vector Machine (SVM) can be used for resume-role classification. A separate matching stage compares candidate information with job requirements. The system is intended to support, not replace, recruiter review. Its evaluation should use real project data and report measured accuracy, precision, recall, F1-score, processing time and ranking behaviour.

Keywords— Resume Screening, Artificial Intelligence, Natural Language Processing, TF-IDF, Support Vector Machine, Candidate Ranking, Job Description

I. INTRODUCTION

Recruitment begins with collecting and reviewing candidate applications. When many resumes are received for the same vacancy, the initial screening stage can consume substantial time. Resume documents also vary in structure: skills may appear as lists, project descriptions, tables or short statements. A screening system therefore needs a repeatable way to extract text and compare it with the requirements of a job.

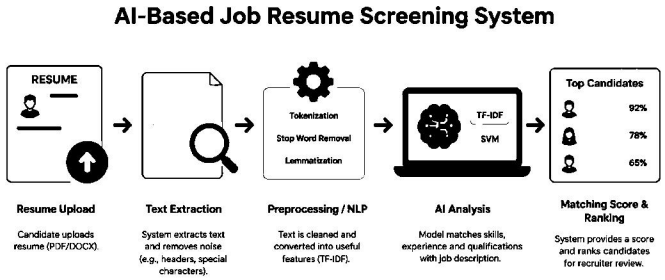
Natural Language Processing provides methods for working with unstructured text. In the proposed project, NLP is used to extract resume text, normalize it, identify relevant terms and prepare features for matching and classification. TF-IDF provides a transparent representation of word importance, while SVM offers a conventional supervised-learning option for classifying resumes into predefined role categories [1], [2].

The goal is not to make the final hiring decision automatically. Instead, the system produces screening information—matched skills, missing skills, a score and a candidate order—so that a recruiter can review the original resume before making a decision.

II. LITERATURE REVIEW

Recent IEEE work has explored automated resume screening with NLP and machine learning. A 2025 IEEE conference paper describes preprocessing with tokenization, stop-word removal and lemmatization, followed by TF-IDF features and SVM classification for predefined job roles [1]. Another IEEE study describes resume sorting using NLP, machine learning and cosine similarity against company requirements [2]. A 2024 IEEE conference paper compares several classifiers for job matching and reports SVM as one of the evaluated models [3].

These studies show a common pipeline: convert resume documents into machine-readable text, extract informative features, compare candidates with job requirements and provide a screening output. The present project follows that general pattern but focuses on a small, understandable implementation suitable for a student project.



II. LITERATURE REVIEW (CONTINUED)

The literature also highlights limitations that matter for practical recruitment systems. Keyword-only matching can miss related expressions. Classification models depend on the quality and representativeness of their training data. Automated screening may also reproduce undesirable patterns if the data or features encode demographic signals. A recent IEEE survey specifically examines bias-aware resume screening and recommends inspecting score distributions, feature influence and ranking behaviour [4].

Approach	Main idea	Typical limitation
Keyword rules	Checks exact terms	Misses related wording
TF-IDF + classifier	Learns from labelled examples	Needs representative training data
NLP extraction	Finds structured resume fields	Can fail on unusual layouts
Similarity matching	Compares resume and job text	Sensitive to representation quality
Proposed system	Combines extraction, features, scoring and ranking	Requires project-level testing

FIG. 1. Comparison of common resume-screening approaches.

III. PROBLEM STATEMENT AND MOTIVATION

The project addresses the first-level screening problem: given a resume and a job description, how can relevant information be extracted and compared in a repeatable way? A useful system should accept common document formats, reduce formatting noise, identify skills and qualifications, and produce an understandable result.

Three practical requirements motivate the design. First, the process should reduce repetitive manual comparison. Second, the system should preserve evidence for the score by showing matched and missing skills. Third, the implementation should remain simple enough to understand, test and modify in a college project.

A. Research Gap

Existing methods often focus on one component, such as classification or keyword matching. The proposed project combines document extraction, NLP preprocessing, feature construction, classification and job-specific matching in one small workflow. The resulting output is intended to be more informative than a single accept/reject label.

B. Objectives

The objectives are to: (1) accept PDF/DOCX resumes; (2) extract and clean text; (3) identify skills, education and experience terms; (4) represent text using TF-IDF; (5) classify resumes with an SVM when labelled role data are available; (6) compare resume features with a selected job description; (7) calculate a transparent match score; (8) rank multiple candidates; and (9) present the result through a simple interface.

IV. PROPOSED SYSTEM

The proposed system has two primary inputs: a candidate resume and a job description. The resume is converted to text and normalized. The job description is processed through the same text-cleaning steps so that both documents can be compared in a common representation.

IV. PROPOSED SYSTEM (CONTINUED)

The processing pipeline separates document handling from scoring. This separation makes the project easier to debug: if the extracted text is incorrect, the problem can be fixed before the classifier or scoring stage is changed.

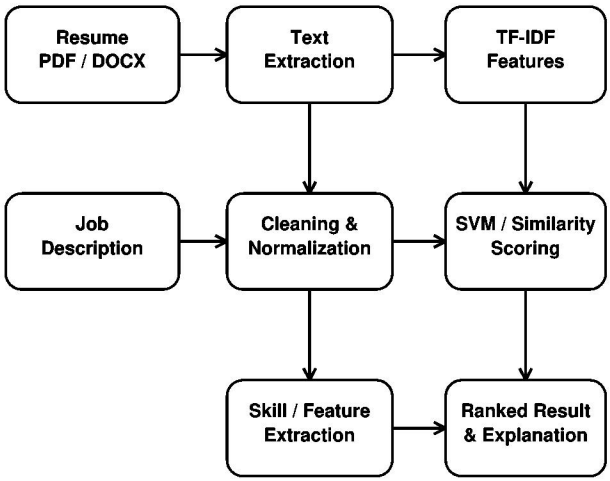


FIG. 2. Original architecture of the proposed resume screening system. The diagram is drawn for this paper and is not copied from an external source.

A. Resume Text Extraction

The file reader converts a PDF or DOCX document into plain text. Headings, bullet points and repeated whitespace are normalized. If the document contains scanned images rather than selectable text, a separate OCR step may be required; the basic student implementation can initially focus on text-based PDF and DOCX files.

B. Text Cleaning

Cleaning can include lowercasing, punctuation handling, whitespace normalization and removal of common stop words. The cleaning stage should not remove terms that carry useful information, such as programming languages, tools, qualifications or job-specific abbreviations.

C. Feature Extraction

TF-IDF converts the cleaned documents into numerical vectors. The representation gives greater weight to terms that are important in a document but less common across the complete document collection. This approach is widely used in information retrieval and text classification [5].

D. Role Classification

When labelled resume categories are available, an SVM classifier can be trained to predict a predefined job-role category. SVMs have a long history in text categorization and can work effectively with high-dimensional sparse feature spaces [6]. The classifier is optional when the project is evaluated only as a matching system.

E. Job-Specific Matching

The resume and job description can be represented with the same TF-IDF vocabulary. Cosine similarity or a weighted feature overlap can then provide a job-specific matching component. The final result can combine this component with explicit skill, education and experience checks.

F. Output

The output screen can show the candidate identifier, overall match score, matched skills, missing skills, detected experience and education terms, and candidate rank. The original resume remains available for human review.

V. SYSTEM IMPLEMENTATION

The implementation is divided into modules so that each stage can be tested independently. A small project can begin with file reading and text extraction before adding classification and the user interface.

1) Input Module. Accepts a resume file and the selected job description.

2) Text Module. Extracts text, removes formatting noise and prepares normalized text.

3) NLP Module. Identifies important terms and structured information such as skills, education and experience.

4) Feature Module. Creates TF-IDF vectors from resume and job-description text.

5) Matching Module. Calculates similarity and explicit skill overlap.

6) Classification Module. Uses SVM to classify a resume into predefined roles when training data are available.

7) Scoring Module. Combines project-defined components into a readable score.

8) Ranking Module. Orders candidates for the selected job.

9) Result Module. Displays matched skills, missing skills and the score.

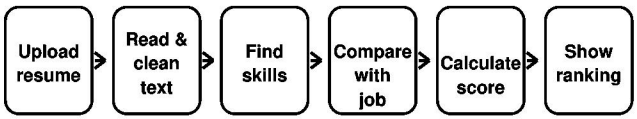


FIG. 3. Original resume-screening process flow.

A. Basic Processing Logic

Input: resume file R and job description J. First, extract text from R and J. Next, normalize both texts and identify useful fields. Then create TF-IDF vectors, calculate job-specific similarity and, where labelled training data exist, obtain the SVM role prediction. Finally, combine the project-defined components, store the screening record and display the result.

B. Candidate Score

A transparent score can be defined as a project configuration rather than a universal hiring rule. One example is shown below:

$$\text{Final Score} = 0.50 \times \text{Skill Match} + 0.20 \times \text{Experience Match} + 0.15 \times \text{Education Match} + 0.15 \times \text{Text Similarity}$$

The weights should be treated as experimental settings. They must be documented and tested rather than presented as a scientifically universal hiring formula.

Module	Input	Output
File reader	PDF / DOCX	Plain text
Text cleaner	Raw text	Normalized text
Feature extractor	Clean text	TF-IDF vectors
Classifier	Feature vector	Role category
Matcher	Resume + job	Similarity
Scorer / ranker	Components	Score + order

FIG. 4. Main implementation modules and their data flow.

VI. MATCHING, SCORING AND RANKING

A resume should not be judged only by whether one exact keyword appears. The matching stage can use explicit skill terms together with a document-level similarity measure. This produces evidence that can be shown to the recruiter.

A. Skill Matching

A project skill dictionary can contain technical skills, tools, frameworks and other terms relevant to the selected job. The system compares detected resume skills with required and preferred skills in the job description. The output can distinguish matched skills from missing skills.

B. TF-IDF Representation

For a term t in document d, TF-IDF is commonly represented as the product of term frequency and inverse document frequency. A simplified form is:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log(N / \text{DF}(t))$$

where N is the number of documents and DF(t) is the number of documents containing term t. The resulting vectors can be compared using cosine similarity [5].

C. Cosine Similarity

If r is the resume vector and j is the job-description vector, cosine similarity can be expressed as:

$$\text{sim}(r,j) = (r \cdot j) / (\|r\| \, \|j\|)$$

This gives a text-similarity component. It should not be interpreted as a complete measure of candidate suitability; it is one signal within the project.

D. Candidate Ranking

After scores are calculated for several resumes against the same job description, candidates can be ordered by score. The interface should show the components of the score so that the ranking can be inspected instead of treated as a black box.

E. Explainable Result

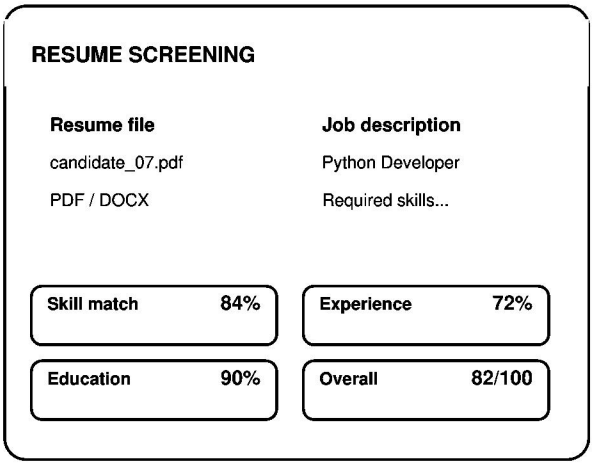


FIG. 5. Original wireframe for the proposed screening result interface.

The wireframe is an original project illustration, not a screenshot or copied application image. It shows where the resume input, job description and score components can be displayed.

TABLE I. EXAMPLE SCORE COMPONENTS

Component	Example role in project
Skill match	Required and preferred skills found in resume
Experience match	Relevant experience terms or duration
Education match	Degree / qualification compatibility
Text similarity	TF-IDF cosine similarity

VII. DATA FLOW AND DATABASE DESIGN

The system can use a small relational database to store only the information needed for screening and evaluation. Separating candidate, job and screening records makes it possible to screen several resumes against one job description without duplicating the full job record.

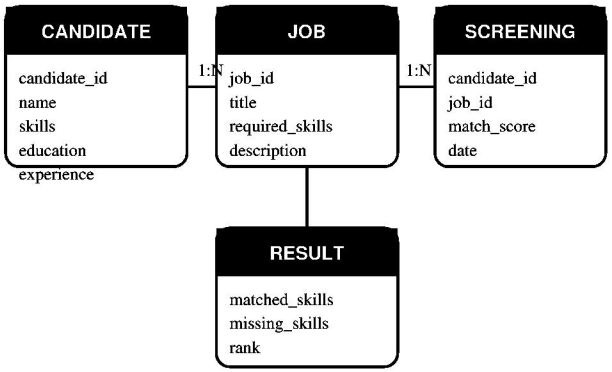


FIG. 6. Original simplified data model for screening records.

A. Candidate Record

A candidate record can contain an internal candidate ID, file name, extracted skills, education terms and experience information. Personal information that is not required for the project should not be collected.

B. Job Record

A job record can contain a job ID, title, required skills, preferred skills and the job-description text. The same job record can be reused when evaluating a group of resumes.

C. Screening Record

A screening record links a candidate to a job and stores the calculated score, matched skills, missing skills, rank and screening date. Keeping the calculated components allows the result to be inspected later.

Record	Example fields
Candidate	Candidate ID, file name, skills, education, experience
Job	Job ID, title, required skills, description
Screening	Candidate ID, job ID, score, rank, date
Result	Matched skills, missing skills, component scores

FIG. 7. Example database fields.

D. Privacy and Fair Use

Resumes can contain names, contact details, education history and other personal information. The prototype should minimize stored data, restrict access to project users and avoid unnecessary collection. Automated screening should remain a decision-support function. The recruiter should review the original application before taking employment action.

E. Bias and Audit Checks

The evaluation can include checks for unexpected score differences when non-job-related attributes are changed or removed. Recent research on automated resume screening shows why bias analysis is relevant to system design [4]. A student project should document the dataset, preprocessing choices and limitations rather than claiming that a model is automatically fair.

VIII. EVALUATION METHOD

The evaluation must use a defined dataset of resumes and job descriptions. Because this paper does not contain measurements from a completed implementation, the figures on this page are **illustrative templates only**; they must be replaced by the actual values obtained during testing.

A. Test Cases

Test cases should include strong matches, partial matches, weak matches, missing-skill cases, different resume formats and resumes containing terms that are related but not identical to job-description wording.

B. Classification Metrics

If labelled role categories are used, report accuracy, precision, recall and F1-score. Accuracy alone can be misleading when the classes are unbalanced, so the class distribution should also be reported.

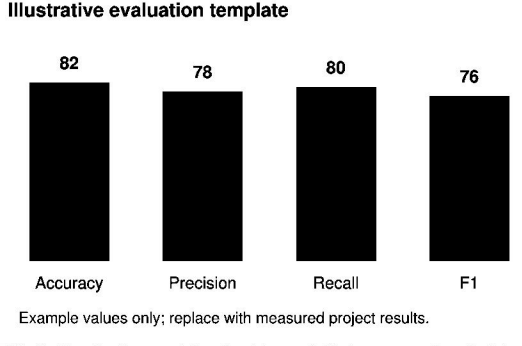


FIG. 8. Illustrative metric chart layout. Values are placeholders, not measured results.

C. Ranking Evaluation

For the ranking component, evaluate whether resumes that satisfy more of the predefined job requirements are placed appropriately. The evaluation should use a documented relevance rule and should not claim that a numerical rank is equivalent to a hiring decision.

D. Processing Time

Measure the time required for file reading, text preprocessing, feature creation and scoring. Report the number of resumes, average processing time and hardware/software environment. This provides a reproducible performance description.

E. Error Analysis

Record failures caused by unusual layouts, scanned documents, missing sections, spelling variations and ambiguous job descriptions. The system should expose extraction failures instead of silently treating unavailable information as absent.

Test	Input	Measure
Text extraction	PDF / DOCX resumes	Successful extraction rate
Role classification	Labelled resumes	Accuracy / F1
Job matching	Resume + JD pairs	Similarity / relevance
Ranking	Multiple candidates	Ranking quality
Performance	Different resume counts	Processing time

FIG. 9. Evaluation plan for the completed implementation.

IX. RESULTS AND DISCUSSION

A completed project should report only values obtained from its own experiments. The discussion should explain not only the final score but also how the system behaves when important skills are added or removed, when resume formatting changes and when several candidates are screened against the same job.

A. Skill-Sensitivity Test

Start with a baseline resume and progressively add or remove required skills. The expected analysis is whether the skill-match component responds in the intended direction. The final paper should include the measured scores from the implementation.

B. Matching Behaviour

Use paired examples where two documents express similar requirements with different wording. Compare the similarity value with a second pair containing overlapping words but different context. This helps determine whether the representation is useful beyond exact keyword overlap.

C. Candidate Ranking

Screen several resumes against one fixed job description. Record each candidate's component scores and final rank. The discussion should identify cases where the ranking behaves unexpectedly and trace the cause to extraction, feature weighting or scoring configuration.

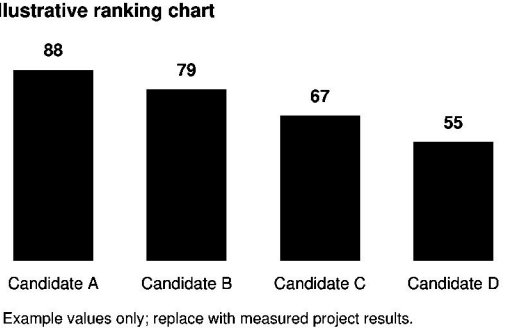


FIG. 10. Illustrative ranking-chart layout; replace with actual project measurements.

D. Processing-Time Analysis

A simple performance experiment can process increasing numbers of resumes and record total runtime. The paper should state the hardware, Python version, libraries and dataset size so that the result can be reproduced.

E. Discussion

The proposed workflow has a practical advantage: the individual stages can be inspected. If the score is unexpectedly low, the developer can examine extracted text, detected skills, TF-IDF features and component scores. This modular design is useful for a college project because errors can be isolated instead of hidden inside one large model.

At the same time, the score should not be treated as a universal measure of candidate quality. Resume content can omit relevant experience, use unusual terminology or contain information that the project does not model. The final decision therefore remains with a human reviewer.

F. Reproducibility

For a final demonstration, preserve the dataset version, preprocessing rules, model parameters, scoring weights and test split. Store evaluation logs separately from candidate personal data. These practices make it easier to explain why two runs produced different results.

X. SYSTEM REQUIREMENTS AND IMPLEMENTATION CHOICES

The project can be implemented with commonly available software. The exact libraries may change during development; the important requirement is that each module produces a clear output for the next stage.

Part	Example choice	Purpose
Language	Python	Application and NLP pipeline
NLP	NLTK / spaCy	Tokenization and text processing
Features	TF-IDF	Numerical text representation
Classifier	SVM / scikit-learn	Role classification
File parser	PDF / DOCX parser	Resume text extraction
Database	SQLite / MySQL	Store screening records
Interface	Simple web interface	Upload and result display
Hardware	Normal laptop	Development and testing

FIG. 11. Example software and hardware requirements.

A. Development Sequence

A practical implementation sequence is: first create the project folder and resume-upload module; second verify PDF/DOCX text extraction; third add cleaning and skill extraction; fourth generate TF-IDF features; fifth train and evaluate the SVM if labelled data are available; sixth add job-description matching; seventh implement score and ranking; and finally connect the user interface and database.

B. Pseudocode

Input: resume R, job description J
1. Extract text from R and J.
2. Clean and normalize both texts.
3. Extract skills, education and experience terms.
4. Create TF-IDF vectors.
5. Calculate text similarity.
6. Predict job category using SVM when trained data exist.
7. Calculate the project-defined score.
8. Store screening details and rank candidates.
9. Display matched skills, missing skills and score.

C. Expected Output

For each screened candidate, the system can show the file name or candidate ID, overall match score, matched skills, missing skills, relevant education and experience terms, predicted role category and rank. The original resume should remain available for final human review.

D. Project Feasibility

The architecture is intentionally modular. A student can first implement file reading and TF-IDF matching, then add SVM classification and the web interface. This staged approach reduces debugging complexity and makes it possible to demonstrate working modules before the complete system is integrated.

XI. LIMITATIONS AND FUTURE WORK

Resume screening is affected by document quality. Different layouts, tables, graphics and scanned pages can make text extraction unreliable. Skill names can also appear in many forms, and a fixed dictionary may fail to recognize a new term.

A second limitation is training data. If the classifier is trained on a small or narrow dataset, its performance may not generalize to other roles, industries or resume styles. Any reported metric must therefore identify the dataset and test population.

A third limitation is the scoring configuration. The weights used in the final score are project choices. They should be validated experimentally and should never be presented as a universal hiring standard.

A. Future Enhancements

Future work can add stronger resume-section detection, more complete skill taxonomies, multilingual processing, improved handling of scanned documents, explainable feature contributions and broader evaluation datasets. A recruiter feedback mechanism can also be studied, provided that feedback is collected carefully and is not allowed to reproduce historical selection patterns without audit.

The system can additionally support multiple job descriptions, candidate-to-job recommendations and exportable screening reports. These additions should be introduced only after the core extraction, matching and evaluation pipeline is reliable.

XII. CONCLUSION

This paper presented an AI-based job resume screening system using conventional NLP and machine-learning techniques. The system accepts resume and job-description documents, extracts and cleans their text, identifies relevant information, constructs TF-IDF features and can use an SVM classifier for predefined job categories. A separate matching and scoring stage produces matched skills, missing skills, a score and candidate ranking.

The design emphasizes transparency and modular implementation. Each stage can be inspected and tested independently, which is appropriate for a student project. The proposed system is a first-level screening aid rather than an automatic final hiring authority. Its effectiveness should be established through real project measurements, documented datasets and reproducible evaluation procedures.

PROJECT SAFETY AND FAIR-USE NOTE

The screening output should be treated as decision support. Personal information should be minimized, stored securely and used only for the stated project purpose. Evaluation should include error analysis and, where feasible, checks for unintended differences associated with attributes that are not job requirements [4].

REFERENCES PREVIEW

The final reference list is provided on the next page. All references are real publications or official publisher/library records and are directly related to resume screening, text classification, NLP or the methods used in the proposed system.

REFERENCES

[1] "AI-Powered Resume Screening System Using NLP and Machine Learning," in *2025 3rd International Conference on Inventive Computing and Informatics (ICICI)*, 2025. IEEE. DOI: 10.1109/ICICI65870.2025.11069785.

[2] "Resume Screening using Machine Learning," in *2023 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI)*, 2023. IEEE. DOI: 10.1109/ICDSAAI59313.2023.10452483.

[3] "Optimizing Job Matching Through Automated Resume Screening with NLP and Machine Learning," in *2024 IEEE International Conference on Intelligent Signal Processing and Effective Communication Technologies (INSPECT)*, 2024. IEEE. DOI: 10.1109/INSPECT63485.2024.10896014.

[4] "Bias-Aware Resume Screening: A Survey and Analytical Framework Using NLP and Machine Learning," in *2026 9th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech)*, 2026. IEEE. DOI: 10.1109/IEMENTech202669403.2026.11434402.

[5] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[6] T. Joachims, "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," in *European Conference on Machine Learning (ECML)*, 1998, pp. 137–142. DOI: 10.1007/BFB0026683.

[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423.

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410.

[10] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

REFERENCE VERIFICATION NOTE

The first four entries are directly related IEEE publications on automated resume screening, job matching and bias-aware resume screening. Entries [5]–[10] support the classical information-retrieval, text-classification and NLP methods used or discussed in the proposed system. The entries were checked against publisher or scholarly records rather than copied from the model paper.

FIGURE AND IMAGE NOTE

All diagrams and charts in this 11-page version are original vector illustrations created specifically for this project paper. No advertisement logos, stock photographs, copied screenshots or figures from the model paper are used.

END OF PAPER